05

AULA 05 · Quarta, 19/08

CI/CD e Ciclo de Vida das Aplicações

Da ideia ao deploy automatizado com qualidade

Módulo · Arquitetura de Serviços e Dados

Prof.: Everton J Benedicto

ada

Cronograma das Aulas

01
Seg 10/08
Introdução ao Módulo e Docker

02
Qui 13/08
Arquitetura de Servidores

03
Sex 14/08
Apache e Nginx

04
Seg 17/08
Proxy Reverso e DNS

05
Qua 19/08
CI/CD e Ciclo de Vida

06
Sex 21/08
Cloud Computing

07
Seg 24/08
Arquitetura de Dados e Data Warehouse

08
Ter 25/08
Data Lake

09
Qua 26/08
Concluir atividades e preencher rubricas

COMBINADO DA TURMA

Por favor!

Bora participar!



Câmera ligada + mão levantada
= aula top!

INTERVALO · 16:30 BRT

Combinados da nossa aula

1. Câmeras ligadas.
2. Entrem no horário: início e intervalo.
3. Participem da aula.
4. Tirem dúvidas.
5. Levantem a mão para tirar dúvidas.

REVIEW!

Recapitulando a Aula 04

- ✓ Proxy reverso protege, distribui carga e faz cache — o cliente nunca acessa o servidor direto.
- ✓ DNS converte nomes amigáveis em endereços IP.
- ✓ Registros: A (IPv4), AAAA (IPv6), MX (e-mail), CNAME (apelido), TXT (verificação).
- ✓ Nginx é o proxy reverso mais popular — simples e de alta performance.

INTEGRAÇÃO CONTÍNUA

CI — Integração Contínua

1. **Commits frequentes:** pequenas mudanças várias vezes ao dia
2. **Build automático:** cada commit compila e gera artefatos
3. **Testes automáticos:** unitários, integração e regressão
4. **Feedback rápido:** erros detectados em minutos, não em dias

💡 ANALOGIA

CI é como cozinhar com um sous chef ao lado: a cada ingrediente ele prova e avisa. Colocou sal demais? Você descobre na hora — não na hora de servir o prato!

Ações e verificações automatizadas a cada commit

O que pode ser feito no Processo de CI?

1. **Análise de Código (Linting & SAST):** padrões de estilo, boas práticas e varredura de vulnerabilidades.
2. **Build & Compilação Automatizada:** empacotamento e geração de artefatos (.jar, imagens Docker).
3. **Testes Automatizados:** testes unitários, de integração e de regressão da funcionalidade.
4. **Análise de Qualidade de Código:** SonarQube para medir cobertura de testes e débito técnico.
5. **Gerenciamento de Dependências & Segurança:** bibliotecas desatualizadas ou com falhas conhecidas (SCA).
6. **Notificação & Feedback Rápido:** alertas instantâneos (Slack, e-mail ou Teams) sobre a compilação.

💡 **REGRA DE OURO DO CI**

Se qualquer uma dessas etapas falhar, a esteira é interrompida imediatamente e o código não avança.
O feedback deve ser gerado em poucos minutos.

ENTREGA CONTÍNUA

CD — Entrega e Deploy Contínuos

1. **Entrega Contínua:** código sempre pronto para produção
2. **Deploy Contínuo:** o que passa nos testes vai automático
3. **Ambientes:** Dev → QA → Staging → Produção
4. **Estratégias:** Blue-Green, Canary e Feature Toggles

💡 ANALOGIA

CD é a linha de montagem de carros: cada peça é testada antes de seguir.
No fim, o carro sai da fábrica pronto para rodar, sem revisão manual a cada parafuso!

ESTRATÉGIAS DE DEPLOY

Implantação sem downtime

1. **Blue/Green Deployment:** dois ambientes idênticos com troca instantânea do tráfego.
2. **Canary Releases:** liberação gradual para uma porcentagem reduzida de usuários.
3. **Rolling Updates:** atualização progressiva das instâncias/pods, sem parar o serviço.

💡 POR QUE IMPORTA

Todas as três estratégias reduzem o raio de impacto de uma falha: se algo der errado, apenas parte do tráfego é afetada e a reversão é imediata.

GITOPS E IAC

Infraestrutura como Código e GitOps

- **Estado desejado declarado em código:** a infraestrutura é versionada como qualquer aplicação.
- **Sincronização automática:** o agente compara o estado real com o declarado e corrige divergências (drift).
- **Ferramentas de IaC:** Terraform, OpenTofu, Ansible e Pulumi.
- **Ferramentas de GitOps:** ArgoCD e FluxCD, com reconciliação contínua em Kubernetes.

💡 DESTAQUE

O repositório Git é a única fonte de verdade da infraestrutura: todo deploy vira um commit auditável e reversível.

PROMOÇÃO DE AMBIENTES

Promoção entre Ambientes e Gates de Aprovação

1. **Dev:** o artefato é construído uma única vez e validado com testes automatizados.
2. **Staging / UAT:** validação funcional e de aceite com dados e configuração próximos da produção.
3. **Produção:** promoção do mesmo artefato, mudando apenas variáveis e segredos por ambiente.
4. **Gates de qualidade:** aprovações manuais, regras de compliance e janelas de manutenção automatizadas.

💡 REGRA DE OURO

Build once, deploy anywhere: o mesmo artefato percorre todos os ambientes — o que muda é apenas a configuração.

BANCO DE DADOS

Migrações de Banco sem Downtime

1. **Expand:** adiciona a nova coluna ou tabela mantendo a estrutura antiga em uso.
2. **Transition:** a aplicação escreve nas duas estruturas enquanto os dados são migrados.
3. **Contract:** a estrutura legada é removida após a migração completa dos dados.

💡 FERRAMENTAS

Flyway e Liquibase versionam e aplicam as migrações dentro do pipeline — o schema evolui junto com o código, de forma controlada.

PROGRESSIVE DELIVERY

Feature Flags e Entrega Progressiva

- **Deploy ≠ Release:** o código vai para produção desligado e é ativado quando o negócio decidir.
- **Toggles em tempo real:** ativar ou desativar recursos sem exigir um novo deploy.
- **Liberação segmentada:** por localização, tipo de usuário, plano ou permissão.
- **Testes A/B:** comparação de variantes com métricas reais de uso.
- **Ferramentas:** LaunchDarkly, Unleash e Flagsmith.

💡 BENEFÍCIO

Separar deploy de release reduz o risco: uma funcionalidade problemática é desligada em segundos, sem pipeline e sem rollback.

VERIFICAÇÃO

Testes em Produção e Verificação Automatizada

- **Smoke tests pós-deploy:** validação automática dos fluxos críticos logo após a publicação.
- **Synthetic monitoring:** robôs simulam jornadas reais de usuário em intervalos regulares.
- **Health probes:** liveness e readiness no Kubernetes controlam tráfego e reinícios.
- **Métricas em tempo real:** latência, taxa de erro e saturação validam a saúde do novo deploy.

💡 IDEIA-CHAVE

Produção é o único ambiente real. Verificar continuamente lá é o que transforma o deploy em uma decisão baseada em dados.

RESILIÊNCIA

Rollback Automatizado e Auto-Healing

1. **Detecção automática:** APM e observabilidade acusam aumento de erros 5xx ou de latência.
2. **Reversão automática:** falha nos testes canary ou nas probes dispara a volta para a versão anterior.
3. **Circuit breaker:** isola dependências instáveis e protege o restante da infraestrutura.
4. **Auto-healing:** instâncias não saudáveis são substituídas automaticamente pelo orquestrador.

> 💡 **META**
>
> *Reduzir o MTTR: importa menos evitar toda falha e mais voltar ao estado saudável em minutos, de forma automática.*

DEVSECOPS

Segurança e Governança no CD

- **Segredos dinâmicos:** HashiCorp Vault e AWS Secrets Manager, sem credenciais no repositório.
- **Menor privilégio:** RBAC e IAM restritos para as service accounts do pipeline.
- **Policy as code:** Open Policy Agent e Kyverno bloqueiam deploys fora da política.
- **Trilha de auditoria:** registro completo de quem aprovou, o que subiu e quando.

💡 GOVERNANÇA

Segurança no CD não é um portão no fim do processo: é um conjunto de verificações automáticas dentro do próprio pipeline.

ECOSSISTEMA

Matriz de Ferramentas de CD

ETAPA DO PIPELINE	FERRAMENTAS
GitOps / Deploy	ArgoCD · FluxCD · Octopus Deploy
Infraestrutura como Código	Terraform · Ansible · Pulumi · ARM Template · CLI
Feature Flags	LaunchDarkly · Unleash
Observabilidade / Canary	Prometheus · Datadog · Flagger · Azure Application Insights
Gestão de Segredos	HashiCorp Vault · AWS Secrets Manager · Azure KeyVault

💡 **COMO ESCOLHER**

Não existe stack única: escolha por maturidade do time, ecossistema de nuvem e nível de automação que a operação consegue sustentar.

CONCLUSÃO

Conclusão e Perguntas

PERGUNTA PARA REFLEXÃO

Sua equipe tem confiança suficiente para fazer deploy na sexta-feira às 17h?

- **Automatize o caminho até produção,** não apenas o build.
- **Meça cada deploy:** frequência, lead time, taxa de falha e tempo de recuperação.
- **Prepare a reversão** antes de precisar dela.

💡 **RESUMO FINAL**

Um pipeline de CD bem estruturado elimina o medo do deploy e transforma lançamentos em eventos rotineiros e de baixo risco.

PIPELINE

O Caminho do Código — Pipeline CI/CD

CI **INTEGRAÇÃO CONTÍNUA** — do commit ao artefato validado

Commit	Pull Request	Build	Testes	Qualidade
push na branch	review + merge	compila e empacota	unitários e integração	SAST, SCA e artefato

artefato aprovado segue para o CD

CD **ENTREGA E IMPLANTAÇÃO CONTÍNUAS** — do artefato validado à produção

Staging / UAT	Gate	Deploy	Produção	Monitorar
validação de aceite	aprovação ou automático	blue-green e canary	release progressivo	métricas e rollback

Feedback contínuo: falhou no CI, o merge é bloqueado; falhou no CD, o rollback é automático se necessário.

Commit → Pull Request → Build → Testes → Qualidade → Staging → Aprovação → Deploy → Produção.

FERRAMENTAS

Ferramentas Populares de CI/CD

1. **Jenkins:** open source, flexível — o clássico do mercado
2. **GitLab CI/CD:** pipeline em YAML dentro do repositório
3. **GitHub Actions:** workflows automáticos direto no GitHub
4. **Azure DevOps:** pipelines, boards e repositórios integrados
5. **ArgoCD:** especialista em Kubernetes — GitOps

💡 ANALOGIA

São assistentes com perfis diferentes: Jenkins é o veterano que faz tudo, GitHub Actions é o moderno integrado ao código, e ArgoCD é o especialista em Kubernetes.

ALM - Application Lifecycle Management

Ciclo de Vida das Aplicações (ALM)

1. **Governança:** requisitos, planejamento e priorização
   Define o que fazer e por quê — viabilidade financeira, portfólio e valor de negócio.
2. **Desenvolvimento:** design, código, CI e testes
   Arquitetura e UX, codificação e automação de testes com integração contínua antes do go-live.
3. **Operações:** monitoramento, incidentes e atualizações
   Infraestrutura, métricas e logs, resolução de incidentes e correções entregues via CD.
4. **Metodologias:** Waterfall, Agile e DevOps
   Do modelo linear aos frameworks ágeis (Scrum, Kanban) até a cultura DevOps.

💡 ANALOGIA

ALM é gerir uma cidade: planejamento urbano (governança), construção das casas (desenvolvimento) e manutenção de ruas e serviços (operações). Todas as fases importam!

RESUMO

Recapitulando a Aula

- ✓ CI = integrar e testar com frequência para achar problemas cedo.
- ✓ CD = manter o código pronto para produção e automatizar o deploy.
- ✓ Pipeline: commit → build → testes → staging → produção.
- ✓ ALM cobre todo o ciclo, do requisito ao monitoramento em produção.